Assignment 5, Team 08

Valentin Oprea (s3906701) Konstantinos Gkaras (s5941814) Caolán Leonard (s6156681)

April 18, 2026

5.1 Reinforcement Learning

(a)

Criteria	Supervised Learning	Reinforcement learning
Learning	Learns from explicitly provided input-output examples, using optimization techniques to find the best model for the given labeled data.	Learns through interactions with the environment by receiving positive/negative reinforcement based on actions taken.
Supervision	Requires external supervision (the labels must be provided).	Does not require supervision. Learns to take appropriate actions to optimize future rewards.
Goal	Getting the smallest error possible between predicted and actual outputs.	Optimizing actions to get the maximal reward possible.

(b)

The robot has a deterministic policy since it has a particular action for each possible state. In this case there are two possible states: either there is a wall in front of the robot, or not. If there is a wall, the robot turns 90 degrees to the right, if there is no wall, move one step up.

(c)

If the robot starts learning with the unsuccessful policy, multiple problems may occur. The robot may constantly choose undesirable actions, making its learning progress very small or non-existent. It might repeatedly get itself into places it can not get outside of, or choose to explore only a very small part, failing to perform its intended task. As a result, it might never learn the desired behavior at all.

(d)

For a state to be considered a Markov state, it must have all the necessary information to predict the next state. Which means that the future state depends only on the current state and not on any of the past states. It must comply with the Markov property: [1]

$$\mathbb{P}(X_{t+1} = s \mid X_t = s_t, X_{t-1} = s_{t-1}, \ldots, X_0 = s_0) = \mathbb{P}(X_{t+1} = s \mid X_t = s_t)$$

(e)

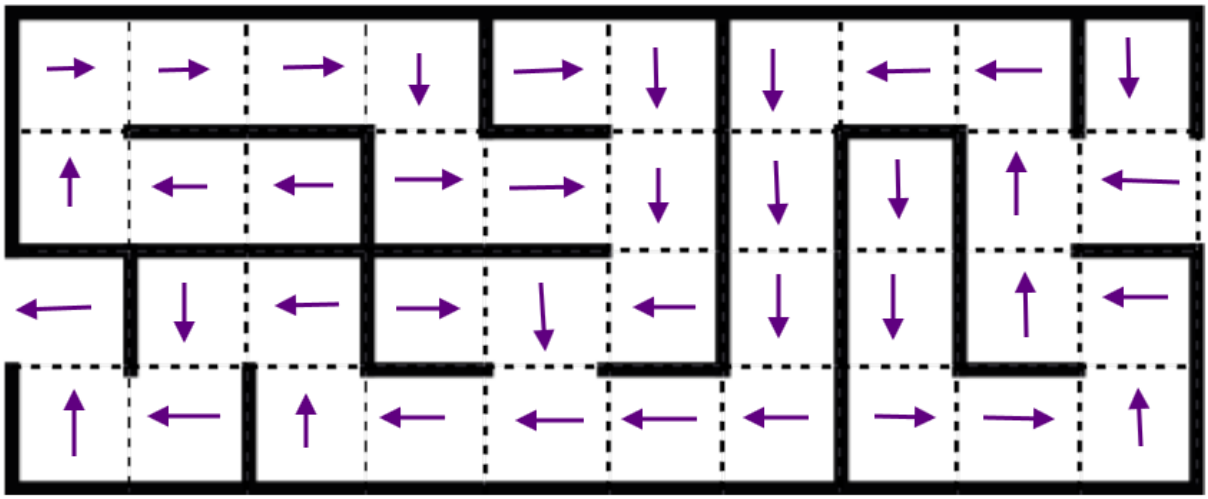
In the case of $\gamma = 0$, the agent makes its decisions short sighted, favouring short-term rewards while ignoring long-term ones. In the case of $\gamma = 1$ the agent considers all future rewards, it plans to maximize the sum of future rewards.

(f)

The agent would be able to solve only this particular maze, since it makes decisions based on states defined by the current position (x, y). In a different maze, the same position (x, y) would correspond to

a different "terrain", making its actions useless. For the agent to be able to solve general mazes, it would need a policy that does not rely on absolute coordinates.

(g)



(h)

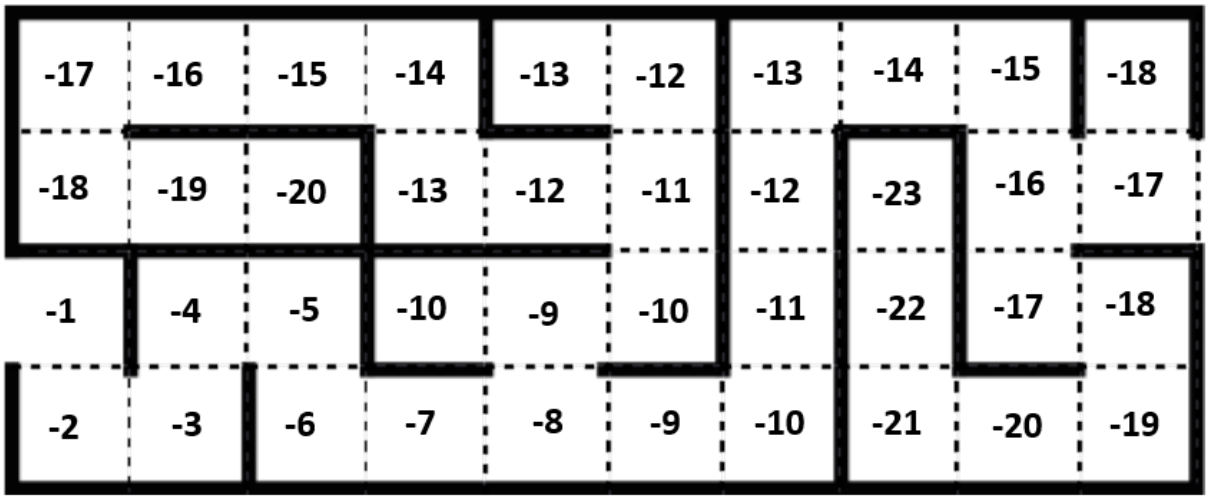


Figure 1: Since the exit is on the left the cumulative reward when the agent exits the maze should be 0.

5.2 A Reinforcement Learning Experiment

Before delving in the solutions and technicalities of this exercise, it is worthwhile to mention that instead of downgrading to the version of the code, we upgraded to the newer version of Gymnasium by OpenAI, based on the officially maintained fork of the old `gym` repository. There are detailed instructions on the `README` file on how to run the code.

Now, with this intricacy out of the way, we provide the answers to the questions. Snippets of the original code and our custom-made modules will be provided when it is necessary, but under no circumstance the full code will be listed here.

(a)

In the updated (slight modifications so it can run on the newest version of the `gymnasium` package) `EnvTest1.py` we have the following line which determines the action of the agent.

```
for i in range(500):
    action = 1
    # env.step updates the environment
```

Listing 1: Lines 9 - 11 of `EntTest1.py` file.

According to the documentation of the specific reinforcement learning environment, the action space consists of 3 actions, namely:

a) 0: Accelerate to the left.
b) 1: Don't accelerate.
c) 2: Accelerate to the right.

Since our action is *don't accelerate*, we live in a trivially deterministic (and ineffective) state.

(b)

Now, we update the code with a stochastic policy, and a deterministic one that is arguably more useful than what we had before. To this end, we introduce a global variable in the `config.py` file named `WORKFLOW`, so we can manually manipulate the flow of our `car.py` script.

```
    if WORKFLOW == "0":
        print("Policy is stochastic (no learning).")
        obs, _ = env.reset(seed=SEED)
        for _ in range(EPISODES):
            action = env.action_space.sample()
            obs, reward, terminated, truncated, _ = env.step(action)
            if RENDER:
                env.render()
            if terminated or truncated:
                break
```

Listing 2: Lines 86 - 95 of `car.py` file, showcasing the stochastic policy

In **Listing 2** we implement a (crude) stochastic policy for the agent. By ignoring all the other commands and focusing on the `action=...` statement, we can see that we randomly pick one of the three possible actions from the action space, and make the agent perform it.

Obviously in this case, our end-game is for the car to reach the flags on top of the mountain. By just performing random movements, we make it very difficult to achieve this goal, mainly because we would have to roll a sequence of appropriate length, where we only take the *accelerate right* action. The probability to generate such a random sequence, when sampling randomly from the action space, is very very small, and can be computed with basic probabilistic arguments.

```
    elif WORKFLOW == "1":
        print("Policy is deterministic (no learning).")
        obs, _ = env.reset(seed=SEED)
        for _ in range(EPISODES):
            action = 2 if obs[1] >= 0 else 0
            obs, reward, terminated, truncated, _ = env.step(action)
            if RENDER:
                env.render()
            if terminated or truncated:
                break
```

Listing 3: Lines 97 - 106 of `car.py` file, showcasing the deterministic policy.

In the contrary, the deterministic policy in **Listing 3** is much more efficient. In this case, we explicitly say to the car driver to *accelerate to the right* if his speed is greater or equal to 0, or if this criterion is satisfied, to *not accelerate at all*. This ensures that at some point, the driver will reach the hill no matter what.

In some cases, this has the unexpected result that the car develops so much speed that it flies past the flags at the top of the hill. This of course is unwanted in a real-life reinforcement learning scheme, but at this point we have achieved our end-game goal, for the first time.

In case you would like to see the animations, and how these two policies perform visually, you can modify the `config.py` file to your heart's content. To be more specific, make sure to turn down the number of `EPISODES` as well as the `MAX_STEPS`. Then, go to `RENDER="human"`, and modify the `WORKFLOW` variable either with `"0"` or `"1"`, to pick between the stochastic or deterministic policies respectively.

It might be evident at this point, that if you would like to modify certain hyperparameters of the `car.py` script, you do so through `config.py`. We have played around with them for a bit to solve the following exercises, so there is not particularly a default setting for each exercise. However, we have ensured that every choice works, so you should get the same results as the ones we list in this report, if you follow the same hyperparameter configuration.

(c)

Playtesting with the stochastic and deterministic policies is nice, but ultimately doesn't make the algorithm learn and respond to the environment. Before we delve into the **Q-learning** scheme, a necessary mathematical analysis must take place.

According once more to the documentation of the Mountain Car environment, the observation space returns each observation as a `ndarray` of shape `(2,)`. Each element corresponds to the following index, type and minimax table:

Index	Observation	Min	Max
0	Position of the car along the $x-$axis	-1.2	0.6
1	Velocity of the car	-0.07	0.07

Table 1: Tabular information regarding the state space of the Mountain Car environment.

As such, mathematically the state space is characterized by

$$S = [p_{\min}, p_{\max}] \times [v_{\min}, v_{\max}] = [-1.2, 0.6] \times [-0.07, 0.07]$$

where p, v denote the position and velocity of the car respectively. Obviously, this is a continuous state space, and in order to implement the **Q-learning** scheme later on, we need to somehow discretize it.

To this end, we go back to the first lecture, and more specifically, to bins. By taking the hint in the next sub-question, we are going to use 19 bins for the position and 15 bins of the velocity. The code supports other configurations as well, but these are the default (and suggested) ones.

```python
def make_grid(low, high, n_bins):
    '''
    Function that makes a uniform grid after the required number of bins has been
    parsed as input. The output grid has size of n_bins-1, this will be useful
    later on.
    '''
    return np.linspace(low, high, n_bins + 1)[1: - 1]
```

Listing 4: Lines 22 - 28 of `car.py` file, showcasing the function that generates a uniform grid based on boundaries and number of bins (internal cut points).

By using the very handy `linspace()` (**Listing 4**) command of the `numpy` package, we create a $N+1$ evenly spaced points between the specified boundaries. Then we strip the first and last one, and return as output an array which holds the $N-1$ bins that we want. This list of cut-points is going to be used later on by a simple map (**Listing 5**), which turns the 2D continuous observation state into a **discrete state** index.

```python
def observation_to_state(obs, pos_grid, vel_grid, n_vel_bins):
    '''
    Turn a 2D grid index (in this case is the position-bin and velocity-bin
    respectively), into a single 1D state index s. Nothing special, just a simple
    map.
    '''
    i = int(np.digitize(float(obs[0]), pos_grid))   # i = 0, ..., POS_BINS-1
    j = int(np.digitize(float(obs[1]), vel_grid))   # j = 0, ..., VEL_BINS-1
    return i * n_vel_bins + j
```

Listing 5: Lines 30 - 38 of `car.py` file, showcasing the simple index mapping that turns the continuous observation space into a discrete state index

Mathematically, these two functions turn the continuous state space into something more machine friendly that can be later used by the **Q-learning** scheme. In addition, the new discrete space has

$$|S_{\text{discrete}}| = n_p \times n_v = 19 \times 15 = 285$$

discrete states (i, j). The index map

$$s = i \times n_v + j$$

helps the algorithm keep track in which of those states it currently is, and by taking this into consideration, it decides on what to do next.

(d)

Since we did most of the heavy lifting regarding dimensions in the previous question, we now just have to initialize the **Q-table**. This is done by following the commands in **Listing 6**, which also print a check in the terminal regarding the final shape of the matrix. Since we have 285 discrete states, and 3 discrete actions we can take, the Q-table belongs

$$Q \in \mathbb{R}^{285\times 3}$$

and is first initialized full of zeros.

```
    # Initialize the Q-table. This is an zeros matrix of shape (285, 3)
    Q = np.zeros((n_states, n_actions), dtype=np.float32)   # ensure float
    print(f"Q-matrix shade:", Q.shape)                      # check shape
```

Listing 6: Lines 79 - 81 of `car.py` file, showcasing the initialization of the Q-table.

(e)

For **(e)**, when the algorithm is given a state s, its next action is chosen from the Q-table, according to the nested if statement:

```
                s = observation_to_state(obs, pos_grid, vel_grid, VEL_BINS)
                if rng.random() < EPSILON:
                    # Explore
                    action = int(rng.integers(n_actions))
                else:
                    # Learn
                    row = Q[s]
                    maxq = row.max()
                    best_actions = np.flatnonzero(row == maxq)
                    action = int(rng.choice(best_actions))
```

Listing 7: Lines 117 - 126 of `car.py` file, showcases the commands to update action based on the Q-table.

Here, we first discretize the continuous observation into a discrete index state. Then, given this discrete state the action is chosen directly from the Q-table's row `Q[s]`. Following this action, we take a maximum over the whole row, and find the index in which it is stored. In case there are many actions that are equivalently good, we take them all into consideration and then pick one of them.

(f)

Now, after executing action a in the discretized state s, the environment gives us back a reward r and the next observation. We map that observation to s' and update the Q-table following the update rule in the Lecture Notes/Wikipedia:

$$Q(s, a) \leftarrow Q(s, a) + a\left[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right]$$

In **Listing 9** we showcase the code that performs this update rule as a function. This later lives inside the main learning-loop of the algorithm, and it constantly updates the table based on observations and rewards from the environment, for each $t = 0, \ldots,$ `MAX_STEPS`, and for each $ep = 0, \ldots,$ `EPISODES`.

```python
def q_update(Q, s, a, r, s_next, done, alpha, gamma):
    '''
    Q-learning update rule drawn from Wikipedia/Lecture Notes.
    '''
    target = r if done else r + gamma * float(Q[s_next].max())
    Q[s, a] += alpha * (target - Q[s, a])
```

Listing 8: Lines 40 - 45 of `car.py` file, showcase the Q-table update rule.

As is indicated in the description of this exercise, we need to also implement a custom stopping condition. This has been on line 133:

```
                # add early success termination if we reached the hilltop
                done = bool(terminated or truncated or success)
```

Listing 9: Line 133 of `car.py` file, showcase the stopping criterion.

This boolean variable decides if the algorithm stopped because it reached the maximum training time, or it stopped because it succeeded or failed. In more details, `truncated` means that the episode ended because the time limit expired, `terminated` means that the episode ended because the agent succeeded or failed on its task, and lastly `success` is a custom variable that only checks if the agent reached the flags on top of the hill.

(g)

To balance between exploration and exploitation, we need to do one final modification to the code and decide if the agent explores or not. Lines 117 to 126 in **Listing 7**, implement this through the use of an if-statement.

Here, we task the algorithm to pick a number in $[0, 1]$. If the number that the algorithm rolled is less than `EPSILON`, then it explores. Otherwise, it chooses the greedy action based on the Q-learning scheme.

This concludes the coding part of this exercise, and the only thing that is left is to run the game and see how it performs for different values of hyperparameters. In **Figure 11** are listed the results when we turn the exploration parameter to $\epsilon = 0.8$. In a more pragmatic approach to this reinforcement learning scheme, the code also generates a plot for the learning curve, a figure which is greatly required on **Section 5.3**. As a showcase, **Figure 2** plots the return per episode and the moving average (window = 20) of the returns for the following set of hyperparameters:

- `EPISODES` = 1000
- `MAX_STEPS` = 500
- `ALPHA` = 0.3
- `GAMMA` = 0.9
- `SEED` = 42
- `EPSILON` = 0.0

5.3 Learning Takes Time

For this question, we run the script `car.py`, setting the parameters in the `config.py` file as required.

(a)

To begin, we use a medium range value for the learning rate to ensure that the proportion of episodes in which the car reaches the top is not insignificant and is also without potential instability present. We then pick a large value for the discount factor so that the car prioritises future rewards rather than a myopic approach (Note that this is kept constant throughout the relevant parts of this question). Next, we use $\epsilon = 0.1$ so that our agent is far more likely to select the best known action so far, rather than mainly selecting random actions.

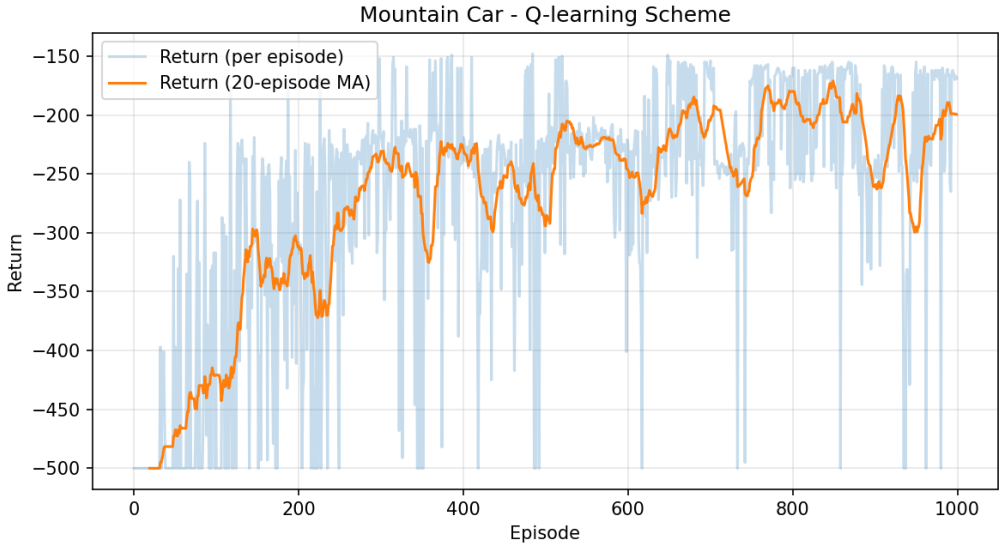


Figure 2: Return per episode & Moving average of the returns plot showcase for **Exercise 5.2**
.

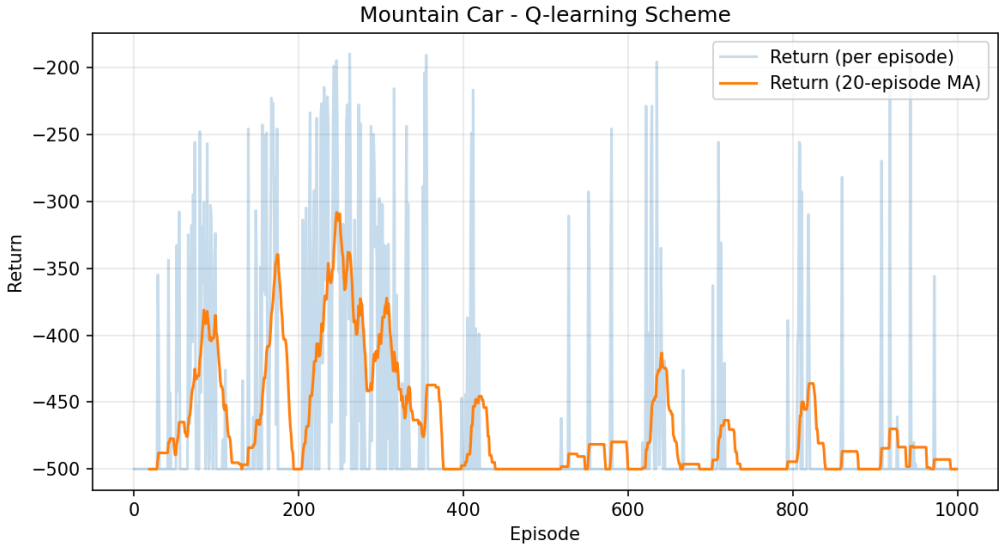


Figure 3: Example with 1000 episodes, max steps is 500.
paramters: $\alpha = 0.5,\ \gamma = 0.9, \epsilon = 0.1.$

The agent's success rate seems to worsen with more episodes. This could be due to the current combination of relatively low learning and exploration rate; the car has found some useful strategies within the first 400 episodes but cannot improve them 'fast enough' via exploration.

(b)

Throughout this part, the following were kept constant:
parameters: $\gamma = 0.9, \epsilon = 0,$ episodes $= 1000,$ max steps $= 500.$

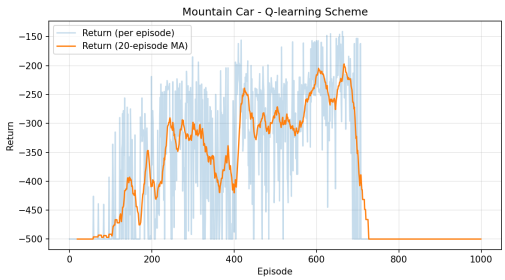


Figure 4: $\alpha = 0.1$

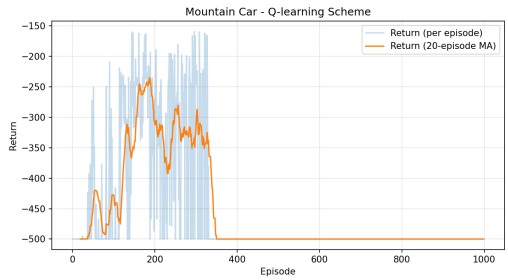


Figure 5: $\alpha = 0.3$

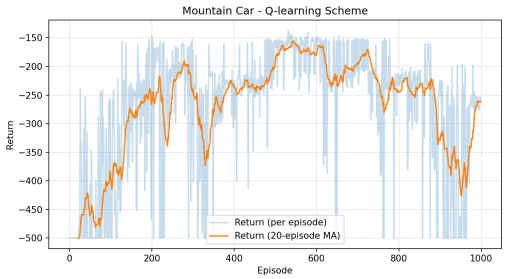


Figure 6: $\alpha = 0.5$

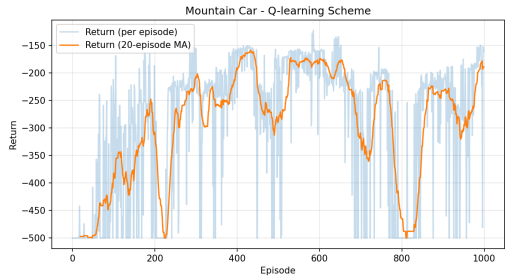


Figure 7: $\alpha = 0.7$

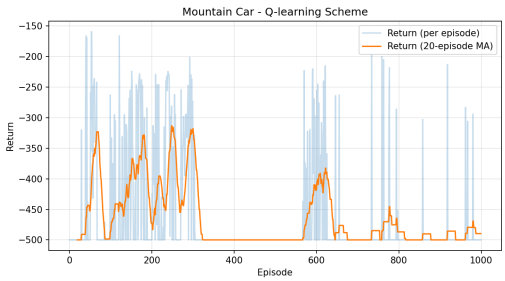


Figure 8: $\alpha = 0.9$

It is first worth mentioning that there does not seem to be a clear outright trend in successful episodes as α is increased. We can note though that the first 'peak' in each figure does seem to increase as the learning rate increases. This could be an indication that initially the learning rate is a dominating parameter. The rapid crash in successful episodes in both 4 and 5 indicates an underlying problem with low values of α as the episode count increases; The car cannot learn fast enough from some repeated failures that occur and as such, remains at the success rate of 0.
The next two figures show much more promising results. Apart from the occasional brief dip in average reward, we can see a general improvement of reward across the simulations for improved learning rates. Finally, in 8 we see that the average rewards more resemble the lower-valued alpha results than they do for the higher-valued ones. The agent seems to struggle in maintaining or improving upon its current optimal reward, possibly because it is too heavily influenced and responsive to new feedback, however useful this new feedback may be. The result is a more erratic profile to the reward per episode. Also the presence of some oscillatory behaviour enforces this notion; the car is 'chopping and changing' too often to figure out an optimal strategy.

(c)

Throughout this part, the following were kept constant:
parameters: $\gamma = 0.9, \alpha = 0.5(\text{best result from (b)}),\ \text{episodes} = 1000,\ \text{max steps} = 500.$

]

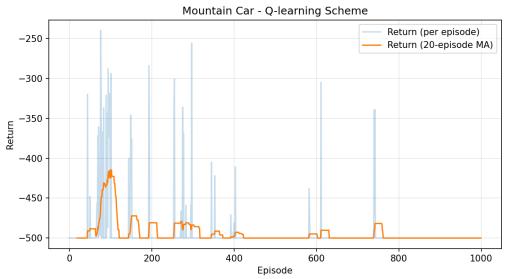


Figure 9: $\epsilon = 0.2$

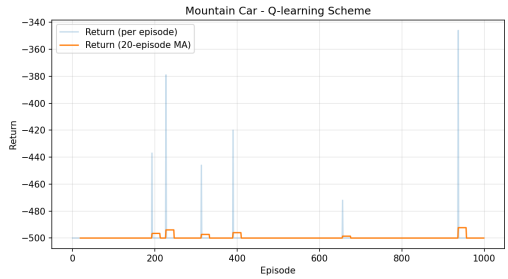


Figure 10: $\epsilon = 0.5$

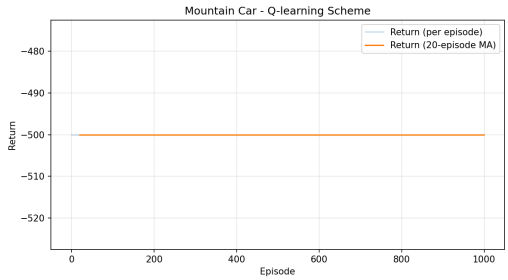


Figure 11: $\epsilon = 0.8$

On an initial glance we immediately notice a clear contrast between results; as ϵ increases we see an overall drastic decline in both the number of successes and peak reward. In fact, for $\epsilon = 0.8$, not one success was registered. We can deduce that as $\epsilon \to 1$, the agent relies more on exploration which is unguided and random. Perhaps without a relatively high learning rate, large ϵ's are not feasible. The car explores many new strategies, potentially some very useful, but cannot learn from them fast enough for progress to be seen. By comparing our first two figures, we can observe that the distribution of reward 'peaks' also becomes more sparse as ϵ increases. Once again, this is likely caused by the car becoming more exploratory so that after a run of successful episodes, it puts less emphasis on learning from its success and more focus on new strategies. As a result, the values of consecutive rewards tend to decrease.

(d)

The last task of this assignment is to tackle the Frozen Lake reinforcement learning environment. This was coded in the `lake.py` module for which simple documentation can be found both in-file and in the `README`.

First of all, at the top of this module are listed the (default) hyperparameters used to solve this task. We didn't want to store them into `config.py` because we don't want to mix them together with the ones for the car. Regarding the specifics of this scheme, the environment has discrete states which take the form of grid cells.

To solve this, we used once again the Q-learning scheme, mainly because we didn't have time to implement something else from scratch. A second reason is also that it works, so why change it? In this setting, the exploration parameter $\epsilon = 0.2$, and once again we pick exploration over exploitation with a probability less than this number.

Regarding training, for each episode the agent interacts with the environment until termination or truncation. Once again, termination means that the agent either died or succeeded, while truncation is that the algorithm run out of time. To evaluate how well the gnome has been trained, we run 500 greedy (no exploration) test episodes and print the result of wins/tests on the terminal. A compact policy map is also printed out, that uses the compass directions for us humans to understand what is going on.

Lastly, some episodes are rendered in a `pygame` window. This animation replays a handful of greedy episodes for visual confirmation by the human. Overall the implementation is quite crude, but probably that was the scope of this exercise. There are many ways in which it could be improved, from increasing the number of episodes, to optimizing the code, and to more advanced reinforcement learning arguments.

References

[1] Rachel Fewster. Lecture notes for stats 325, June 2014. Lecture notes for STATS 325, University of Auckland. Last updated: 20 June 2014. Please send comments to Rachel Fewster.